

System Architecture Document

LKA System — aBAJA SAEINDIA 2026 | Software Virtual World Simulation

1. System Overview

This document describes the software architecture of the Lane Keep Assist (LKA) system developed for the aBAJA SAEINDIA 2026 Software Virtual World Simulation event. The system runs inside IPG CarMaker 15.0 and communicates with the simulator via the CarMaker Python APO (Asynchronous Process Object) interface. Perception is driven entirely by a Camera RSI sensor; no ground-truth lane data or DVA lane quantities are used as algorithm inputs.

Key specifications:

Parameter	Value
Simulator	IPG CarMaker 15.0
Rendering	MovieNX
Camera sensor	Camera RSI — 640×480 RGB, ~30 Hz over TCP port 2210
Target speed	30 km/h
Control loop rate	30 Hz
Vehicle model	Demo_IPG_CompanyCar (wheelbase 2.622 m, mass 1 391 kg)
Language	Python 3 with OpenCV, NumPy, IPG CarMaker Python API

2. Module Architecture

The stack is decomposed into five Python modules, each with a single responsibility. Dependencies flow strictly downward — the main loop imports all other modules but no module imports the main loop, keeping the architecture testable and modular.

Module	Role	Key Outputs
LKA_Main.py	Main control loop — bridges CarMaker and the perception/control stack	Socket initiates APO (acquires camera frames, sets lane search, look-ahead offset, and control parameters)
sensor_fusion.py	Computer vision module. Applies BEV perspective warp, Sliding Window intersection, polynomial fitting, and computes lane parameters	Intermediate lane parameters
config.py	Single source of truth for all tunable parameters: camera resolution, BEV scale (meters), lane width (m), per-metre, PID gains, etc.	Configuration constants
lka_utils.py	Background utility module: RMSE accumulator, DNF watchdog (triggers DNF flag, lka_on_x5s), and CS	RMSE, DNF flag, lka_on_x5s, CS
camera_tcp_client.py	TCP socket client that connects to the Camera RSI sensor stream on port 2210. Handles frame decoding, buffer	Raw 640x480 (Handy frame)

3. Data Flow

Each control cycle (nominally 1/30 s) executes the following pipeline:

Step	Module	Description
1	camera_tcp_client	Receive raw 640×480 RGB frame from Camera RSI over TCP
2	sensor_fusion	Convert to grayscale → apply binary threshold

3	sensor_fusion	Warp to Bird's-Eye View (source trapezoid → top-down rectangle)
4	sensor_fusion	Bottom-half histogram → find left/right lane base positions
5	sensor_fusion	Sliding window search upward → collect lane pixel coordinates
6	sensor_fusion	Fit 1st-order polynomial to each lane → compute lane centre
7	sensor_fusion	Compute lateral error = lane centre – image centre (metric)
8	sensor_fusion	Compute lookahead offset at 5.0 m ahead on fitted lane
9	LKA_Main	Pure Pursuit: compute baseline steering angle from lookahead geometry
10	LKA_Main	PID: compute correction term from lateral error
11	LKA_Main	Sum and rate-limit → clamp to ± 0.20 rad road-wheel angle
12	LKA_Main	Write road-wheel angle to CarMaker APO (DVA write)
13	lka_utils	Log telemetry to CSV; update RMSE accumulator; check DNF watchdog

4. Perception Pipeline — BEV + Sliding Window

The perception approach was selected after evaluating four alternative methods (CLAHE + Percentile Threshold, Hough Transform, Adaptive Thresholding, and Frangi Ridge Detection), all of which failed due to near-black camera frames in the default CarMaker exposure configuration. After correcting the camera settings (ISO 800, F/2.8, Filmic tone mapping), the BEV + Sliding Window pipeline was validated as the most robust classical approach.

4.1 Bird's-Eye View Transform

A perspective warp maps a trapezoidal region of the front-facing camera image to a top-down rectangle. This eliminates perspective foreshortening so that both lane lines appear as near-vertical strips regardless of distance from the camera. The pixels-per-metre calibration ratio converts pixel offsets to metric lateral errors.

4.2 Sliding Window Lane Search

The bottom-half column histogram identifies the starting x-positions of the left and right lanes. A stack of windows (default: 9 windows, margin ± 50 px) searches upward, re-centring on the mean of detected white pixels. This naturally connects dashed lane segments and handles curved roads without angle assumptions.

4.3 Polynomial Fitting and Gap Bridging

A first-order polynomial is fitted to the detected lane pixels in each window stack. The lane centre and lookahead point are derived from the mean of the two fitted curves. Polynomial fits are cached for up to 15 frames, allowing the controller to maintain stable steering through brief detection gaps (dashed line gaps, partial occlusion).

5. Control Architecture — Pure Pursuit + PID

A standalone PID controller reacts to lateral error after accumulation and has no knowledge of vehicle geometry or road curvature, making it prone to oscillation on curves. Pure Pursuit provides a geometrically grounded baseline: it computes the exact steering angle required to follow an arc from the vehicle's current position to a lookahead point on the detected lane, using the vehicle wheelbase and steering ratio.

Parameter	Value	Rationale
-----------	-------	-----------

Pure Pursuit lookahead distance	5.0 m	Provides ~0.6 s preview at 30 km/h; balances responsiveness and smoothness
Vehicle wheelbase	2.622 m	Demo_IPG_CompanyCar specification
Steering ratio	16:1	Vehicle model parameter — maps road-wheel to steering wheel
Max road-wheel angle	± 0.20 rad	Physical steering limit enforced in software
Rate limit	0.03 rad/frame	Prevents actuator saturation and jerky motion
PID Kp / Ki / Kd	0.10 / 0.005 / 0.08	Tuned empirically; Ki prevents steady-state offset; Kd damps overshoot

6. Safety Mechanisms

Mechanism	Implementation	Behaviour
DNF watchdog	lka_utils.py	If front axle exits lane boundaries for >5 consecutive seconds, logs DNF event and terminates
Steering rate limiter	LKA_Main.py	Output delta capped at 0.03 rad per frame regardless of commanded value — prevents high-frequency oscillations
Angle saturation clamp	LKA_Main.py	Final road-wheel command clamped to ± 0.20 rad
Fit cache fallback	sensor_fusion.py	Uses last valid polynomial fit for up to 15 frames if no lanes detected — prevents erratic steering
TCP reconnect	camera_tcp_client.py	Automatically reconnects to Camera RSI on socket drop with exponential backoff

7. Logged Outputs

All run outputs are written to `results/LKA/`:

File	Description
Data/Logs/lka_run.csv	Per-frame telemetry: timestamp, lateral error, steering angle, speed, yaw rate
debug_frames/	Per-frame debug images (BEV view, sliding window visualisation) — archived post-run
debug_raw_frame.png	Last raw camera frame received from CarMaker
debug_bev.png	Last Bird's-Eye View frame after warp
debug_sliding_window.png	Last frame with sliding window overlay and lane fit
debug_vision.png	Last annotated front-view frame with lane centre and error overlay